COS 214: Practical 4 - TaskForge: Hierarchical Work Processing

Rico Geerkens (u25015304)
Declan Mc Cullough (u21760022)
Joshua Peter (u23626527)
GitHub Repo: https://github.com/Rico101Playz/cos214-prac4

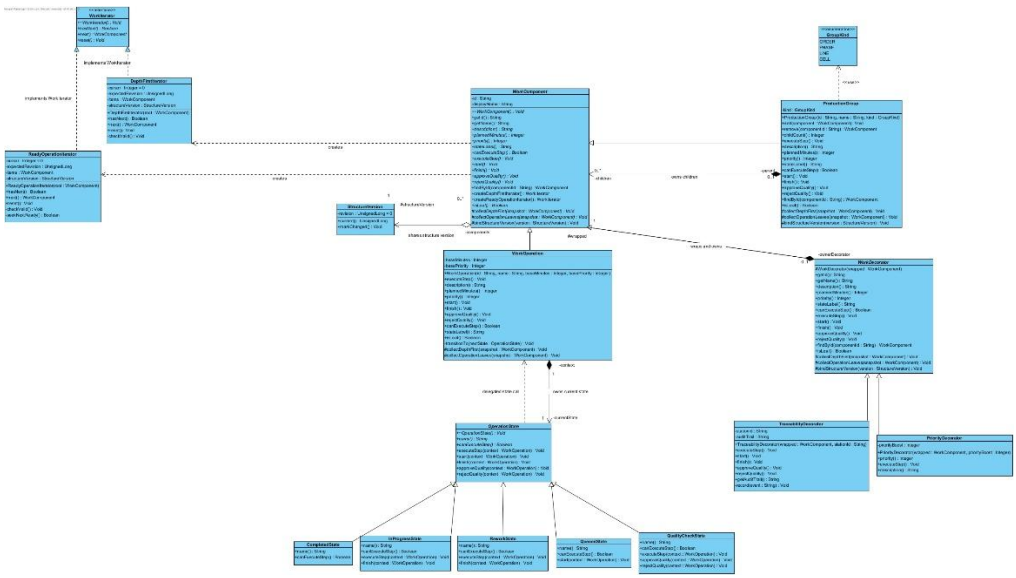
Task 1: Design the System

Chosen Domain: Discrete Manufacturing

TaskForge helps coordinate the work required to produce a product order. A production order is decomposed into production phases, phases into work cells and work cells into physical individual manufacturing processes such as cutting, assembly, drilling, painting and inspection.
This system solves 4 main problems:

- Can be used to represent a recursive production plan without special case-for-case client logic.
- That plan can then be traversed without exposing a group's internal *vector*
- An operation's behaviour can be changed as it moves through production and quality control
- Add optional, stackable specifications to an operation without multiplying subclasses.

Complete UML class diagram:



Identification of the GoF Patterns and participants:

Pattern	Participant	ClassName
Composite	Component	WorkComponent
Composite	Leaf	WorkOperation
Composite	Composite	ProductionGroup
Composite	Client	main.cpp
Iterator	Iterator	WorkIterator
Iterator	ConcreteIterator	DepthFirstIterator
Iterator	ConcreteIterator	ReadyOperationIterator
Iterator	Aggregate	WorkComponent
Iterator	ConcreteAggregate	ProductionGroup
State	Context	WorkOperation
State	State	OperationState
State	ConcreteState	• QueuedState • InProgressState • QualityCheckState • ReworkState • CompletedState
Decorator	Component	WorkComponent
Decorator	ConcreteComponent	WorkOperation
Decorator	Decorator	WorkDecorator
Decorator	ConcreteDecorator	• PriorityDecorator • TraceabilityDecorator

Design Rationale:

One composite class for all group levels
Orders, phases, lines, and cells are all different in their identity and placement, not in the way they own and coordinate work. The *GroupKind* value holds the distinction without four behaviour-less subclasses. Therefore, new group levels can easily be introduced without changing traversal or client code.

Explicit ownership
Any operation that involves ownership (such as adding, removing, moving, decoration) transfers a *unique_ptr*. This prevents shared structural ownership and makes destruction predictable and supports memory safe recursive cleanup.

Leaf-focused decorators
Priority and traceability are policies that are attached to specific shop-floor operations. Decorating leaf chains preserves the same component abstraction while avoiding ambiguity such as whether decorating a group decorates its current children, future children or both.

Task 3: Dynamic Behaviour and Design Decisions

Traversal Modification Policy:
We decided to use an invalidation policy for existing traversals when the structure it refers to changes. When a traversal is created it records the current revision of the structure, an iterator remembers the structural version when it was created and any structural changes increments the structure version. Each iterator checks the current version before performing traversal operations and if the versions differ, the iterator considers itself invalid and throws a logic error.

We selected to use invalidation because when the hierarchy changes, continuing to use old traversal could result in following a path that no longer corresponds to the current production structure. Rather than allowing the iterator to continue with old information, TaskForge detects the change and stops the traversal.

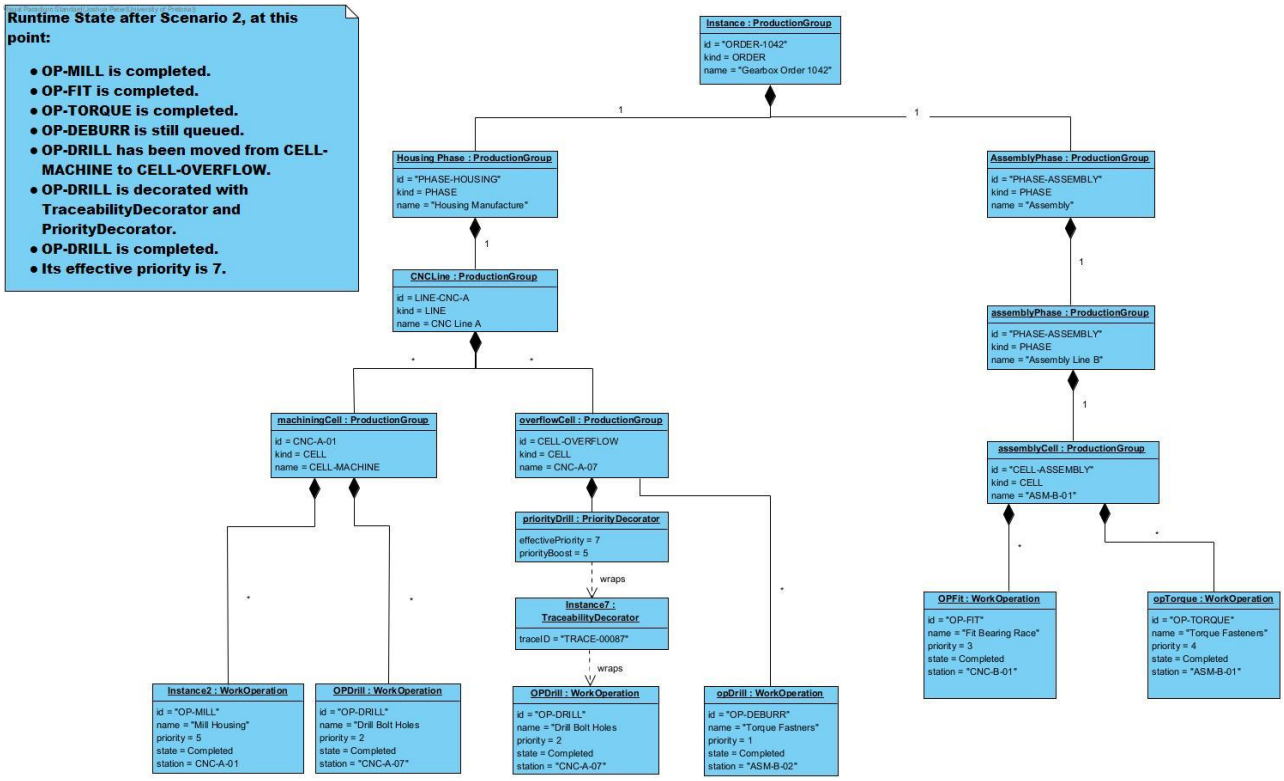
The old iterator is never reused after invalidation, the client discards it and creates a new iterator using the updated hierarchy. This policy is demonstrated in Scenario 2. A depth-first iterator is active when “OP-DRILL” is removed from the machining cell and added to the overflow cell, the existing iterator detects this revision and reports that it has been invalidated. The system creates a new traversal, which reflects the new location of “OP-DRILL”.

Design Decisions:

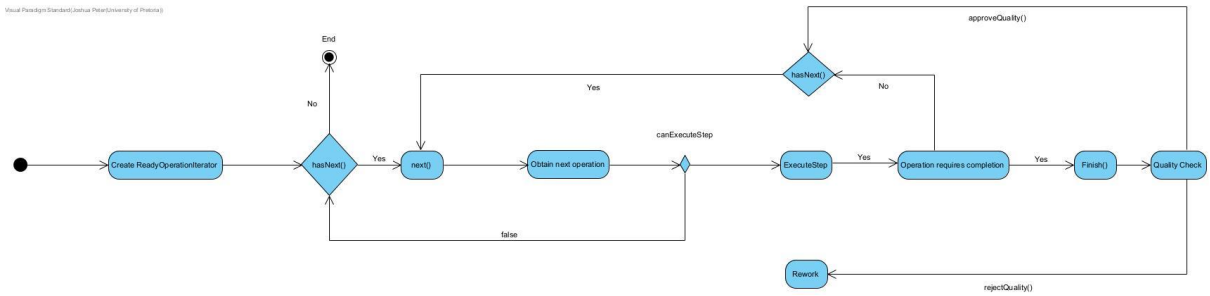
TaskForge manages the work required to manufacture product orders. A production order is represented as a recursive hierarchy in which a production order contains production phases, phases contain production lines, lines contain work cells and work cells contain individual manufacturing operations. The implementation uses the Composite design pattern to represent the hierarchy, Iterator to traverse it, State to control the life cycles and behaviour of individual operations, and Decorator to add responsibilities to operations dynamically.

Task 4: UML Diagram Portfolio

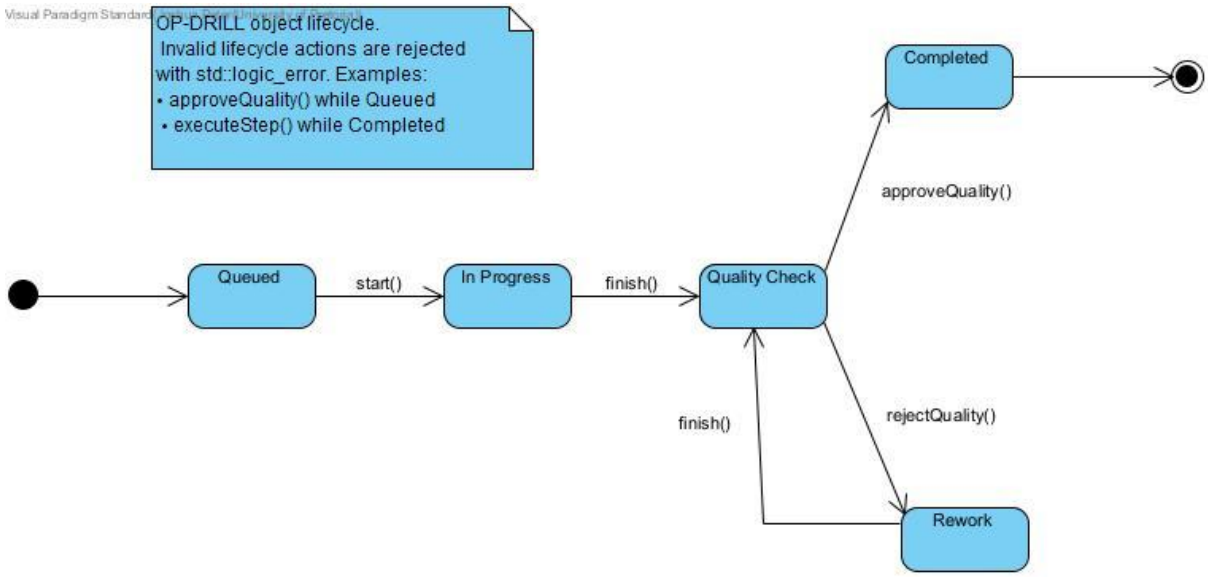
1. UML object diagram showing a meaningful runtime portion of your nested hierarchy

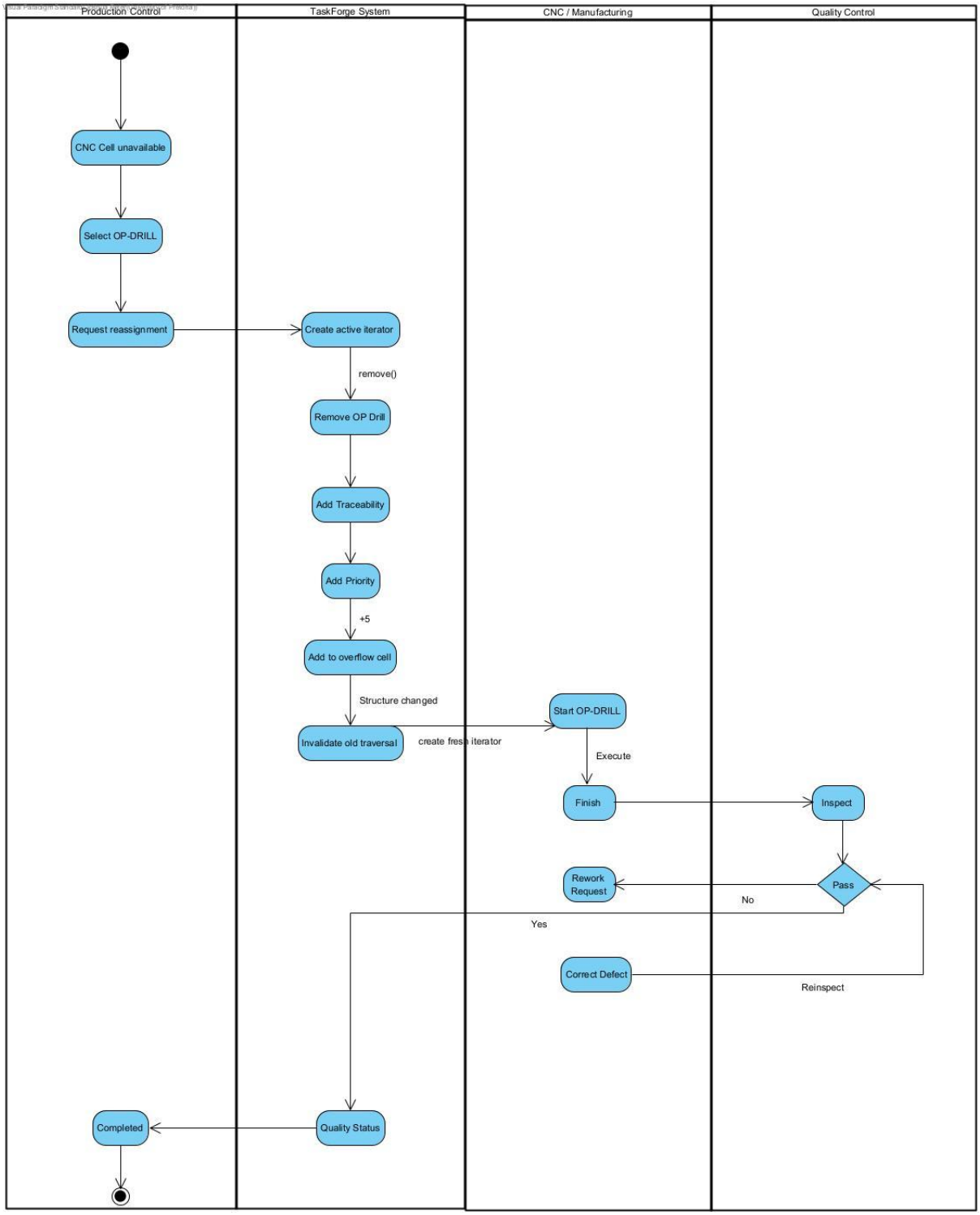


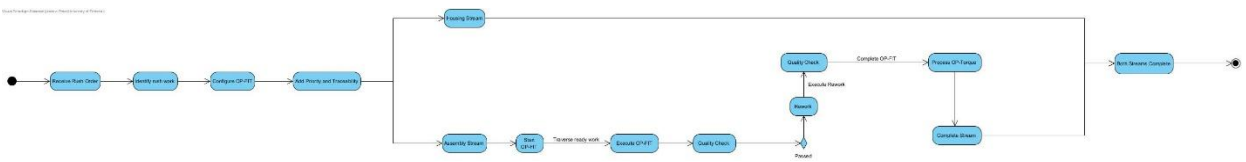
2. one UML state diagram showing the lifecycle of an important stateful object



3. three substantial UML Activity Diagrams chosen to explain the most important workflows in your system




Production Control
TaskForge System
CNC / Manufacturing
Quality Control
CNC Cell unavailable
Select OP-DRILL
Request reassignment
Create active iterator
remove()
Remove OP Drill
Add Traceability
Add Priority
+5
Add to overflow cell
Structure changed
Invalidate old traversal
create fresh iterator
Start OP-DRILL
Execute
Finish
Inspect
Pass
Rework Request
No
Yes
Correct Defect
Reinspect
Completed
Quality Status



Task 5: Debugging and Memory Investigation

```
Reading symbols from ./taskforge...
(gdb) break DepthFirstIterator::DepthFirstIterator
Breakpoint 1 at 0x479f: file DepthFirstIterator.cpp, line 11.
(gdb) run
Starting program: /app/taskforge
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".


============================================================


TaskForge manufacturing system


============================================================
Production order: Production order Gearbox Order 1042 (2 direct children)
Total planned work: 260 minutes
Highest effective priority before runtime changes: 4


============================================================


Independent depth-first traversals


============================================================


Breakpoint 1, DepthFirstIterator::DepthFirstIterator (this=0x555555588350, root=...) at DepthFirstIterator.cpp:11
11          expectedRevision_(structureVersion_->current()) {
(gdb) next
8           : items_(),
(gdb)
9           cursor_(0U),
(gdb)
10          structureVersion_(root.structureVersion()),
(gdb)
11          expectedRevision_(structureVersion_->current()) {
(gdb)
12          root.collectDepthFirst(items_);


(gdb)
13      }
(gdb) print items_.size()
$1 = 13
(gdb) catch throw
Catchpoint 2 (throw)
(gdb) continue
Continuing.


Breakpoint 1, DepthFirstIterator::DepthFirstIterator (this=0x5555555883a0, root=...) at DepthFirstIterator.cpp:11
11          expectedRevision_(structureVersion_->current()) {
(gdb)
Continuing.
First iterator advances twice:
  ORDER-1042       | Production order Gearbox Order 1042 (2 direct children)          | priority=4  | Group with 2 direct components
  PHASE-HOUSING    | Production phase Housing Manufacture (1 direct children)         | priority=3  | Group with 1 direct components
Second iterator still begins at the root:
  ORDER-1042       | Production order Gearbox Order 1042 (2 direct children)          | priority=4  | Group with 2 direct components
First iterator continues from its own cursor:
  LINE-CNC         | Production line CNC Line A (2 direct children)                   | priority=3  | Group with 2 direct components


============================================================


Complete depth-first hierarchy


============================================================


Breakpoint 1, DepthFirstIterator::DepthFirstIterator (this=0x555555588350, root=...) at DepthFirstIterator.cpp:11
11          expectedRevision_(structureVersion_->current()) {
(gdb)
Continuing.
  ORDER-1042       | Production order Gearbox Order 1042 (2 direct children)          | priority=4  | Group with 2 direct components
  PHASE-HOUSING    | Production phase Housing Manufacture (1 direct children)         | priority=3  | Group with 1 direct components
  LINE-CNC         | Production line CNC Line A (2 direct children)                   | priority=3  | Group with 2 direct components
  CELL-MACHINE     | Work cell Machining Cell (2 direct children)                     | priority=3  | Group with 2 direct components
  OP-MILL          | Operation Mill gearbox housing [Queued]                          | priority=3  | Queued
  OP-DRILL         | Operation Drill mounting holes [Queued]                          | priority=2  | Queued
  CELL-OVERFLOW    | Work cell Overflow CNC Cell (1 direct children)                  | priority=1  | Group with 1 direct components
  OP-DEBURR        | Operation Deburr housing [Queued]                                | priority=1  | Queued
  PHASE-ASSEMBLY   | Production phase Gear Assembly (1 direct children)               | priority=4  | Group with 1 direct components
  LINE-ASSEMBLY    | Production line Assembly Line 2 (1 direct children)              | priority=4  | Group with 1 direct components
  CELL-ASSEMBLY    | Work cell Gear Fitment Cell (2 direct children)                  | priority=4  | Group with 2 direct components
  OP-FIT           | Operation Fit gear train [Queued]                                | priority=4  | Queued
  OP-TORQUE        | Operation Torque housing bolts [Queued]                          | priority=3  | Queued


============================================================


Scenario 1: Normal Gearbox Production


============================================================
Starting the housing and assembly work streams.
  [state] OP-MILL: Queued -> In Progress
  [state] OP-FIT: Queued -> In Progress


Ready-operation traversal dispatches executable work:


============================================================


Ready-operation traversal


============================================================
Selected OP-MILL in state In Progress
  [production] Performing Mill gearbox housing (90 minutes planned)
Selected OP-FIT in state In Progress
  [production] Performing Fit gear train (70 minutes planned)


The completed manufacturing steps now enter quality control.
  [state] OP-MILL: In Progress -> Quality Check
  [state] OP-MILL: Quality Check -> Completed
  [state] OP-FIT: In Progress -> Quality Check
  [state] OP-FIT: Quality Check -> Completed


The assembly line continues with the torque operation.
  [state] OP-TORQUE: Queued -> In Progress


============================================================
```

```
Ready-operation traversal


=========================================================
Selected OP-TORQUE in state In Progress
  [production] Performing Torque housing bolts (30 minutes planned)
  [state] OP-TORQUE: In Progress -> Quality Check
  [state] OP-TORQUE: Quality Check -> Completed


=========================================================


Scenario 1 result


=========================================================
  OP-MILL          | Operation Mill gearbox housing [Completed]                  | priority=3  | Completed
  OP-FIT           | Operation Fit gear train [Completed]                        | priority=4  | Completed
  OP-TORQUE        | Operation Torque housing bolts [Completed]                  | priority=3  | Completed


Normal production work has completed successfully.


=========================================================


SCENARIO 2: CNC disruption, reassignment and quality failure


=========================================================


Breakpoint 1, DepthFirstIterator::DepthFirstIterator (this=0x555555588350, root=...) at DepthFirstIterator.cpp:11
11          expectedRevision_(structureVersion_->current()) {
(gdb)
Continuing.
An active traversal begins inspecting the order:
  ORDER-1042       | Production order Gearbox Order 1042 (2 direct children)          | priority=4  | Group with 2 direct components
  PHASE-HOUSING    | Production phase Housing Manufacture (1 direct children)         | priority=3  | Group with 1 direct components


The mounting-hole operation is reassigned because...
 The main CNC cell is unavailable.
  OP-DRILL was moved to the overflow CNC cell.
  Effective priority is now 7.


Checking the traversal that was active during the reassignment:


Catchpoint 2 (exception thrown), 0x00007ffff7df535a in __cxa_throw () from /lib/x86_64-linux-gnu/libstdc++.so.6
(gdb) bt
#0  0x00007ffff7df535a in __cxa_throw () from /lib/x86_64-linux-gnu/libstdc++.so.6
#1  0x0000555555558ad9 in DepthFirstIterator::checkValid (this=0x555555588350) at DepthFirstIterator.cpp:38
#2  0x0000555555558941 in DepthFirstIterator::hasNext (this=0x555555588350) at DepthFirstIterator.cpp:18
#3  0x0000555555566997 in runScenarioTwo (order=..., machiningCell=..., overflowCell=...) at main.cpp:306
#4  0x000055555556f75 in main () at main.cpp:576
(gdb) frame 1
#1  0x0000555555558ad9 in DepthFirstIterator::checkValid (this=0x555555588350) at DepthFirstIterator.cpp:38
38                  "Depth-first iterator invalidated by a structural change");
(gdb) frame 2
#2  0x0000555555558941 in DepthFirstIterator::hasNext (this=0x555555588350) at DepthFirstIterator.cpp:18
18          checkValid();
(gdb) print cursor_
$2 = 2
(gdb) print items_.size()
$3 = 13
(gdb)
```

The code was run but faulted so then it was rerun with gdb in a container but un-beknown to the team at the time, the cache from earlier testing was causing fails in the current testing so after going through gdb and not finding errors the container was reset and cleared of cache which then ended up with a perfectly fine output and no faults.

test1.cpp > ...

```cpp
#include <iostream>
#include <memory>
#include "ProductionGroup.h"
#include "WorkOperation.h"
#include "TraceabilityDecorator.h"
#include "WorkIterator.h"
//testing decorator for proper traversal if a whole group is decorated instead of just a work operation
int main()
{
    std::unique_ptr<ProductionGroup> root(new ProductionGroup("ROOT", "Root", GroupKind::ORDER));
    std::unique_ptr<ProductionGroup> cell(new ProductionGroup("CELL-A", "Cell A", GroupKind::CELL));
    cell->add(std::unique_ptr<WorkComponent>(new WorkOperation("OP-1", "Op One", 10, 1)));
    cell->add(std::unique_ptr<WorkComponent>(new WorkOperation("OP-2", "Op Two", 10, 1)));
    std::unique_ptr<WorkComponent> tracedCell(new TraceabilityDecorator(std::move(cell), "STATION-X"));
    root->add(std::move(tracedCell));
    std::cout << "Total planned minutes: " << root->plannedMinutes() << " (should be 20)\n";
    std::cout << "DF traversal:\n";
    std::unique_ptr<WorkIterator> it = root->createDepthFirstIterator();
    int count = 0;
    while (it->hasNext())
    {
        WorkComponent &c = it->next();
        std::cout << "  " << c.getId() << " - " << c.description() << "\n";
        count++;
    }
    std::cout << "Nodes visited: " << count << " (should be 4 from count:  the root(ROOT), the cell(CELL-A), OP-1 and OP-2)\n";
    return 0;
}
```

(gdb) break DepthFirstIterator::next
Breakpoint 1 at 0x4937: file DepthFirstIterator.cpp, line 23.
(gdb) break WorkDecorator::collectDepthFirst
Breakpoint 2 at 0xf40c: file WorkDecorator.cpp, line 99.
(gdb) break ProductionGroup::collectDepthFirst
Breakpoint 3 at 0x877a: file ProductionGroup.cpp, line 164.
(gdb) break ProductionGroup::collectOperationLeaves
Breakpoint 4 at 0x8846: file ProductionGroup.cpp, line 175.

(gdb) run
Starting program: /app/test1
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Total planned minutes: 20 (should be 20)
DF traversal:

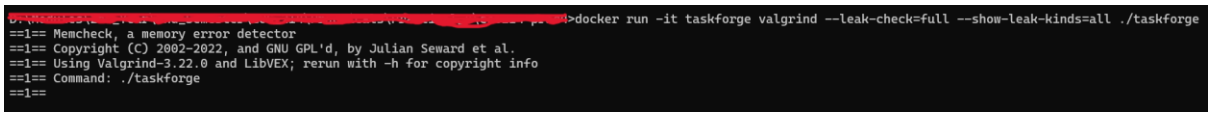
Breakpoint 3, ProductionGroup::collectDepthFirst (this=0x5555555812b0, snapshot=std::vector of length 0, capacity 0) at ProductionGroup.cpp:164
164 std::vector<WorkComponent*>& snapshot) {
The root begins its traversal and collectDepthFirst of the Production group was called.
(gdb) bt
#0 ProductionGroup::collectDepthFirst (this=0x5555555812b0, snapshot=std::vector of length 0, capacity 0) at ProductionGroup.cpp:164
#1 0x000055555555881a in DepthFirstIterator::DepthFirstIterator (this=0x555555581a60, root=...) at DepthFirstIterator.cpp:12
#2 0x00005555555626a8 in WorkComponent::createDepthFirstIterator (this=0x5555555812b0) at WorkComponent.cpp:33
#3 0x0000555555564415 in main () at test1.cpp:18
(gdb) continue
Continuing.

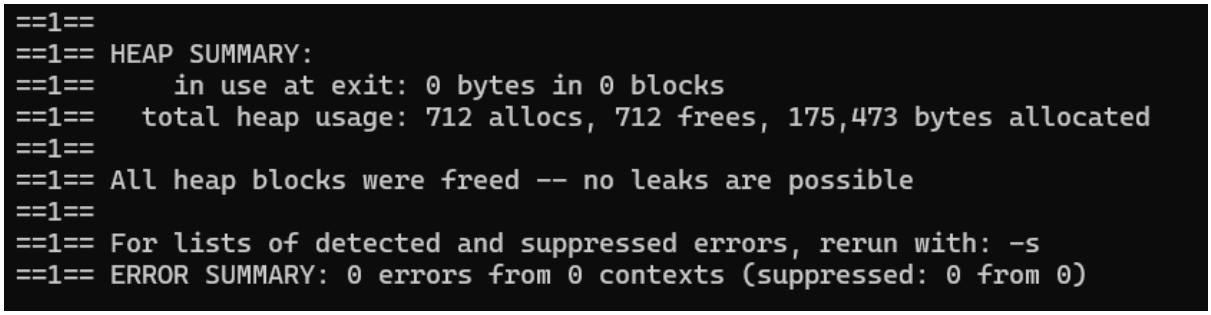
Breakpoint 2, WorkDecorator::collectDepthFirst (this=0x5555555815b0, snapshot=std::vector of length 1, capacity 1 = {...}) at WorkDecorator.cpp:99
99 std::vector<WorkComponent*>& snapshot) {
The added work decorator also has its collectDepthFirst called showing that traversal so far is fine.
(gdb) bt
#0 WorkDecorator::collectDepthFirst (this=0x5555555815b0, snapshot=std::vector of length 1, capacity 1 = {...}) at WorkDecorator.cpp:99
#1 0x000055555555c7e4 in ProductionGroup::collectDepthFirst (this=0x5555555812b0, snapshot=std::vector of length 1, capacity 1 = {...}) at ProductionGroup.cpp:170
#2 0x000055555555881a in DepthFirstIterator::DepthFirstIterator (this=0x555555581a60, root=...) at DepthFirstIterator.cpp:12
#3 0x00005555555626a8 in WorkComponent::createDepthFirstIterator (this=0x5555555812b0) at WorkComponent.cpp:33
#4 0x0000555555564415 in main () at test1.cpp:18
(gdb) continue
Continuing.

Breakpoint 1, DepthFirstIterator::next (this=0x555555581a60) at DepthFirstIterator.cpp:23
23 checkValid();
Still fine the next is called of the Iterator
(gdb) bt
#0 DepthFirstIterator::next (this=0x555555581a60) at DepthFirstIterator.cpp:23
#1 0x0000555555564442 in main () at test1.cpp:22
(gdb) continue
Continuing.
ROOT - Production order Root (1 direct children)

Breakpoint 1, DepthFirstIterator::next (this=0x555555581a60) at DepthFirstIterator.cpp:23
23 checkValid();
This is where the problem reared its head. There is no call of the ProductionGroup's collectDepthFirst inside the decorator, the Iterator simply moves onto next inside the root, meaning the two leaves (Work Operations) are never calling their collectDepthFirst
(gdb) bt
#0 DepthFirstIterator::next (this=0x555555581a60) at DepthFirstIterator.cpp:23
#1 0x0000555555564442 in main () at test1.cpp:22
(gdb) continue
Continuing.
CELL-A - Work cell Cell A (2 direct children) {traceable at STATION-X}
Nodes visited: 2 (should be 4 from count: the root(ROOT), the cell(CELL-A), OP-1 and OP-2)
This is what originally sent alarm bells through our heads as running the test outside the gdb meant the final result did not add up
You will also notice that there is no call upon collectOperationLeaves in the Production group which also shows the depthFirstIterator never enters the Decorator to collect the Leaves.

[Inferior 1 (process 147) exited normally]
A fix was made to only have the Leaves be decorated (WorkOperation) or else there would be a problem in the future.





Task 6: Docker and GitHub Workflow

Task 6.1 - Reflection Paragraph

The work was divided into 3 parts, 1 part per team member. Rico Geerkens took upon himself to do task 1 and task 2, developing the model for our system and creating the UML Class Diagram and writing a rationale for the team. This can be seen by the commits made by Rico first. Following this Joshua Peter took up doing task 3 and task 4. Joshua did the majority of the coding of the system while also completing a UML Diagram Portfolio with 3 Activity Diagrams . He followed up Rico's commits with his own. Declan McCullough then did task 5 and task 6, the debugging of the code using gdb, where problems were met and fixed, as well as integrating Docker, a makefile and a README. The team worked efficiently together to complete the system in the given time frame. Seen by adding commits with changes and added files.